Module 1: Formal Syntax and Semantics

C Program Formatter Written in Java

As our case study for this module, we have chosen a program written in Java that contains a hand-coded scanner and a recursive descent parser. The purpose of this program is to format a C program. The formatting that is to be performed involves properly indenting statements according to their depth of nesting, providing uniform and consistent spacing between tokens, and writing the formatted program to an output file suitable for printing on a fixed-size page. To achieve that requirement, the program lines must not exceed a specified number of characters, and there must be a maximum number of lines per page.

A feature that indents programs automatically is incorporated in many current integrated development environments. In such new environments, the indentation is often done as the program is input. Some environments allow the user to specify how the braces will be placed, where blank lines will be inserted, and so on.

To illustrate how this program will behave, we provide a sample C program that determines the first four perfect numbers, which will be input to the formatter. That program is shown below:

```
#include <stdio.h>
#define NUMBER_TO_FIND 4
void main()
{
int candidate=2, numberFound = 0, sumOfFactors, divisor;
while (numberFound < NUMBER_TO_FIND)
{
sumOfFactors = 0;
for (divisor = 1; divisor <= candidate / 2; divisor++)
if (candidate % divisor == 0)
sumOfFactors += divisor;
if (candidate == sumOfFactors)
```

```
{
printf("%d\n", candidate);
numberFound++;
}
candidate++;
}
}
```

We have intentionally used very poor formatting so that the effect of the formatter will be clearer. When the complete formatter program is run with the above file as input, the output looks as follows:

```
primes
PAGE 1

#include <stdio.h>
#define NUMBER_TO_FIND 4

primes
PAGE 2
void main()

{
int candidate = 2, numberFound = 0, sumOfFactors, divisor;

while (numberFound < NUMBER_TO_FIND)
{
sumOfFactors = 0;
for (divisor = 1; divisor <= candidate / 2; divisor++)
if (candidate % divisor == 0)
sumOfFactors += divisor;
if (candidate == sumOfFactors)
{
printf("%d\n", candidate);
numberFound++;
```

```
}
candidate++;
}
}
```

Next, let's examine the code for this program, beginning with the class that contains the main method. As with any well-designed object-oriented program, the main method is quite short. It creates three *singleton objects*—the scanner, the parser, and the output object—that constitute the program. The code for the main class is shown below.

```
import java.io.*;

class Main
{

// The main function for the C formatter program. It creates
// the three primary objects, an output object, a lexer object,
// and a formatter object. It then calls the file method of the
// formatter object to perform the formatting.

private static final BufferedReader stdin =
new BufferedReader(new InputStreamReader(System.in));

public static void main(String[] args) throws IOException
{
String fileName;
System.out.print("Enter file name without .c: ");
fileName = stdin.readLine();
Output output = new Output(fileName);
Lexer lexer = new Lexer(fileName, output);
Format format = new Format(lexer, output);

format.file();
lexer.close();
```

```
output.close();
}
}
```

As we discussed in the module commentary, the tokens are best defined using an enumerated type, which is what we have elected to do. That enumerated type is shown below:

```
// Token Definitions

enum Token {NOT_FOUND, UPPER_CASE_IDENTIFIER, CONSTANT, COMMENT,
COMPILER_DIRECTIVE, ASSIGNMENT_OPERATOR,
PRE_OR_POST_UNARY_OPERATOR, STRUCTURE_OPERATOR,
UNARY_OR_BINARY_OPERATOR,UNARY_OPERATOR, BINARY_OPERATOR,
TERNARY_OPERATOR, COLON, LEFT_PARENTHESIS, RIGHT_PARENTHESIS,
LEFT_BRACKET, RIGHT_BRACKET, LEFT_BRACE, RIGHT_BRACE, SEMICOLON,
COMMA, STRING, SC_SPECIFIER, BREAK, CASE, TYPE_SPECIFIER,
CONTINUE, DEFAULT, DO, ELSE, FOR, GOTO, IF, RETURN, SIZEOF,
STATUS, STRUCT, SWITCH, UNION, WHILE, IDENTIFIER, FIRST, NONE,
END_OF_FILE}
```

Next, let's examine the class that defines the scanner object. This scanner is hand-coded, rather than being generated by a lexical analyzer that has been provided the regular expression definitions of the tokens. As you will see, most of the methods are private. The primary public method is getNextToken, which is called repeatedly by the parser.

```
// The Lexer class contains a lexical analyzer that returns Tokens
// from the input file on each call to getNextTokens. The class
// also contains functions that adjust the spacing based on the
// type and context of the Tokens.

import java.io.*;

class Lexer
```

```
{
public static final int SUPPRESS_NEITHER_SPACE = 0,
SUPPRESS_LEADING_SPACE = 1, SUPPRESS_TRAILING_SPACE = 2;
private int i = 0, spacing;
private char character;
private String line = "";
private BufferedReader file;
private Output output;
private Token currentToken, lastToken;
private String currentLexeme, lastLexeme;

// Constructor initializes private data members and opens the input
// file.

public Lexer(String fileName, Output output)
throws FileNotFoundException
{
file = new BufferedReader(new FileReader(fileName + ".c"));
character = nextChar();
currentLexeme = "";
lastToken = Token.NONE;
this.output = output;
}

// Closes input file

public void close() throws IOException
{
file.close();
}

// adjustSpacing will set bits in the spacing word to indicate the
// type of spacing adjustment to be done, LEADING OR TRAILING.

public void adjustSpacing(int spacingValue)
```

```
{
spacing |= spacingValue;
}

// checkDeclarationSpacing sets leading or trailing space bits in
// the variable "spacing" according to the type of current token.

public void checkDeclarationSpacing(Token current)
{
if (currentToken == Token.UNARY_OR_BINARY_OPERATOR)
spacing |= SUPPRESS_TRAILING_SPACE;
else if (currentToken == Token.LEFT_PARENTHESIS)
spacing |= SUPPRESS_LEADING_SPACE;
}

// checkExpressionSpacing sets leading or trailing space bits in
// the variable "spacing" according to the types of current and
// last tokens.

public void checkExpressionSpacing(Token current, Token previous)
{
if (current == Token.PRE_OR_POST_UNARY_OPERATOR)
if (previous == Token.IDENTIFIER ||
previous == Token.RIGHT_BRACKET ||
previous == Token.RIGHT_PARENTHESIS)
spacing |= SUPPRESS_LEADING_SPACE;
else
spacing |= SUPPRESS_TRAILING_SPACE;
else if (current == Token.UNARY_OR_BINARY_OPERATOR)
if (previous != Token.IDENTIFIER &&
previous != Token.RIGHT_BRACKET
&& previous != Token.RIGHT_PARENTHESIS && previous !=
Token.PRE_OR_POST_UNARY_OPERATOR)
spacing |= SUPPRESS_TRAILING_SPACE;
}
```

```
// getNextToken returns the next token in the input file and
// displays the previous token. Comment and preprocessor tokens
// are skipped.

public Token getNextToken()
{
if (lastToken != Token.NONE)
{
currentToken = lastToken;
lastToken = Token.NONE;
return currentToken;
}
output.outputToken(currentLexeme, spacing);
spacing = SUPPRESS_NEITHER_SPACE;
lastLexeme = currentLexeme;
do
{
currentLexeme = "";
while (character != 0 && Character.isWhitespace(character))
character = nextChar();
if (character == 0)
{
output.endLine(false);
return Token.END_OF_FILE;
}
if (Character.isUpperCase(character))
{
while (Character.isLetter(character) ||
Character.isDigit(character) || character == '_')
{
currentLexeme += character;
character = nextChar();
}
currentToken = Token.UPPER_CASE_IDENTIFIER;
```

```
}
else if (Character.isLetter(character) || character == '_')
{
while (Character.isLetter(character) ||
Character.isDigit(character) || character == '_')
{
currentLexeme += character;
character = nextChar();
}
currentToken = testToken(currentLexeme);
}
else if (Character.isDigit(character))
{
while (Character.isLetter(character) ||
Character.isDigit(character) || character == '.')
{
currentLexeme += character;
character = nextChar();
}
currentToken = Token.CONSTANT;
}
else if ((currentToken =
testOperator()) != Token.NOT_FOUND)
;
else if ((currentToken =
testSeparator()) != Token.NOT_FOUND)
;
else
currentToken = Token.NOT_FOUND;
}
while (currentToken == Token.COMMENT ||
currentToken == Token.COMPILER_DIRECTIVE);
return currentToken;
}
```

```
// Puts back the last token that was gotten.

public void putLastToken()
{
lastToken = currentToken;
}

// Returns the lexeme corresponding to the last token.

public String getLastLexeme()
{
return lastLexeme;
}

// Returns the next character in the input buffer.

private char nextChar()
{
try
{
if (line == null)
return 0;
if (i == line.length())
{
line = file.readLine();
i = 0;
return '\n';
}
return line.charAt(i++);
}
catch (IOException exception)
{
return 0;
}
}
```

```
// testOperator will return the token type if it is an operator.
// Otherwise, it returns NOT_FOUND. Spacing is set for some
// operators. Comments are ignored.

private Token testOperator()
{
char lastCharacter;

switch(character)
{
case '+':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else if (character == '+')
{
currentLexeme += character;
character = nextChar();
return Token.PRE_OR_POST_UNARY_OPERATOR;
}
else
return Token.UNARY_OR_BINARY_OPERATOR;
case '-':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
```

```
return Token.ASSIGNMENT_OPERATOR;
}
else if (character == '-')
{
currentLexeme += character;
character = nextChar();
return Token.PRE_OR_POST_UNARY_OPERATOR;
}
else if (character == '>')
{
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE |
SUPPRESS_LEADING_SPACE;
return Token.STRUCTURE_OPERATOR;
}
else
return Token.UNARY_OR_BINARY_OPERATOR;
case '*':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else
return Token.UNARY_OR_BINARY_OPERATOR;
case '%':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
```

```
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
case '>':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.BINARY_OPERATOR;
}
else if (character == '>')
{
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
case '<':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
```

```
character = nextChar();
return Token.BINARY_OPERATOR;
}
else if (character == '<')
{
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
case '&':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else if (character == '&')
{
currentLexeme += character;
character = nextChar();
return Token.BINARY_OPERATOR;
}
else
return Token.UNARY_OR_BINARY_OPERATOR;
```

```
case '|':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else if (character == '|')
{
currentLexeme += character;
character = nextChar();
return Token.BINARY_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
case '=':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.BINARY_OPERATOR;
}
else
return Token.ASSIGNMENT_OPERATOR;
case '!':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
```

```
return Token.BINARY_OPERATOR;
}
else
{
spacing = SUPPRESS_TRAILING_SPACE;
return Token.UNARY_OPERATOR;
}
case '/':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
if (character == '*')
{
currentLexeme += character;
character = nextChar();
do
{
lastCharacter = character;
character = nextChar();
}
while (character != '/' || lastCharacter != '*');
character = nextChar();
return Token.COMMENT;
}
else
return Token.BINARY_OPERATOR;
case '~':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE;
```

```
return Token.UNARY_OPERATOR;
case '.':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE |
SUPPRESS_LEADING_SPACE;
return Token.STRUCTURE_OPERATOR;
case '?':
currentLexeme += character;
character = nextChar();
return Token.TERNARY_OPERATOR;
default:
return Token.NOT_FOUND;
}
}

// testSeparator will return the token type if it is a separator,
// otherwise, it returns NOT_FOUND. Compiler directives are
// printed out as they are found.

private Token testSeparator()
{
switch (character)
{
case ':':
currentLexeme += character;
character = nextChar();
return Token.COLON;
case '(':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE;
return Token.LEFT_PARENTHESIS;
case ')':
currentLexeme += character;
```

```
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.RIGHT_PARENTHESIS;
case '[':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE | SUPPRESS_TRAILING_SPACE;
return Token.LEFT_BRACKET;
case ']':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.RIGHT_BRACKET;
case '{':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE;
return Token.LEFT_BRACE;
case '}':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.RIGHT_BRACE;
case ';':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.SEMICOLON;
case ',':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.COMMA;
case '#':
while (character != '\n' && character != 0)
```

```
{
currentLexeme += character;
character = nextChar();
}
output.endLine(false);
output.outputDirective(currentLexeme);
return Token.COMPILER_DIRECTIVE;
case '\'':
currentLexeme += character;
character = nextChar();
while (character != '\'')
{
if (character == '\')
{
currentLexeme += character;
character = nextChar();
}
currentLexeme += character;
character = nextChar();
}
currentLexeme += character;
character = nextChar();
return Token.CONSTANT;
case '"':
currentLexeme += character;
character = nextChar();
while (character != '"')
{
if (character == '\')
{
currentLexeme += character;
character = nextChar();
}
currentLexeme += character;
character = nextChar();
```

```
}
currentLexeme += character;
character = nextChar();
return Token.STRING;
default:
return Token.NOT_FOUND;
}
}

// testToken will return the token type if it is a token.
// Otherwise,it returns IDENTIFIER.

private Token testToken(String lexeme)
{
switch (lexeme.charAt(0))
{
case 'a':
if (lexeme.equals("auto"))
return Token.SC_SPECIFIER;
else
return Token.IDENTIFIER;
case 'b':
if (lexeme.equals("break"))
return Token.BREAK;
else
return Token.IDENTIFIER;
case 'c':
if (lexeme.equals("case"))
return Token.CASE;
else if (lexeme.equals("char"))
return Token.TYPE_SPECIFIER;
else if (lexeme.equals("continue"))
return Token.CONTINUE;
else
return Token.IDENTIFIER;
```

```
case 'd':
if (lexeme.equals("default"))
return Token.DEFAULT;
else if (lexeme.equals("double"))
return Token.TYPE_SPECIFIER;
else if (lexeme.equals("do"))
return Token.DO;
else
return Token.IDENTIFIER;
case 'e':
if (lexeme.equals("else"))
return Token.ELSE;
else if (lexeme.equals("entry"))
return Token.SC_SPECIFIER;
else if (lexeme.equals("extern"))
return Token.SC_SPECIFIER;
else
return Token.IDENTIFIER;
case 'f':
if (lexeme.equals("for"))
return Token.FOR;
else if (lexeme.equals("float"))
return Token.TYPE_SPECIFIER;
else
return Token.IDENTIFIER;
case 'g':
if (lexeme.equals("goto"))
return Token.GOTO;
else
return Token.IDENTIFIER;
case 'i':
if (lexeme.equals("if"))
return Token.IF;
else if (lexeme.equals("int"))
return Token.TYPE_SPECIFIER;
```

```
else
return Token.IDENTIFIER;
case 'l':
if (lexeme.equals("long"))
return Token.TYPE_SPECIFIER;
else
return Token.IDENTIFIER;
case 'r':
if (lexeme.equals("register"))
return Token.SC_SPECIFIER;
else if (lexeme.equals("return"))
return Token.RETURN;
else
return Token.IDENTIFIER;
case 's':
if (lexeme.equals("short"))
return Token.TYPE_SPECIFIER;
else if (lexeme.equals("sizeof"))
return Token.SIZEOF;
else if (lexeme.equals("static"))
return Token.SC_SPECIFIER;
else if (lexeme.equals("status"))
return Token.STATUS;
else if (lexeme.equals("struct"))
return Token.STRUCT;
else if (lexeme.equals("switch"))
return Token.SWITCH;
else
return Token.IDENTIFIER;
case 't':
if (lexeme.equals("typedef"))
return Token.SC_SPECIFIER;
else
return Token.IDENTIFIER;
case 'u':
```

```
if (lexeme.equals("union"))
return Token.UNION;
else if (lexeme.equals("unsigned"))
return Token.TYPE_SPECIFIER;
else
return Token.IDENTIFIER;
case 'v':
if (lexeme.equals("void"))
return Token.TYPE_SPECIFIER;
else
return Token.IDENTIFIER;
case 'w':
if (lexeme.equals("while"))
return Token.WHILE;
else
return Token.IDENTIFIER;
default:
return Token.IDENTIFIER;
}
}
}
```

The next class is the Format class, which defines the formatter. It uses a recursive descent parser. It repeatedly calls the scanner to get another token as needed, and it calls the output object to specify indentation and to output tokens and new lines when appropriate.

The methods to format the if, while, do, and for statements have been omitted. Your instructor may elect to assign you a programming project that entails completing those methods so you will understand better the process of recursive descent parsing.

You should note that the code provided here will not work properly on the sample C program that determines the first four perfect numbers because the formatting for the if, while, for, and do statements is not implemented. Once you have added

this code to the Format class, the case-study example will perform the correct formatting.

```
// The Format class contains the methods necessary for formatting a
// C program. Only the file method is public. Recursive descent
// parsing is used to parse the program and perform the formatting.

class Format
{
private Lexer lexer;
private Output output;
private Token token;

// The constructor establishes the input lexer and the output
// private data members.

public Format(Lexer lexer, Output output)
{
this.lexer = lexer;
this.output = output;
}

// file is the only public method. External
// declarations are formatted until a function is found, at
// which time functionBody is called to format it.

public void file()
{
token = lexer.getNextToken();
while (token != Token.END_OF_FILE)
if (externalDeclaration())
functionBody();
}

// functionBody formats the declarations and statements in a
```

```
// function body.

private void functionBody()
{
output.endLine(true);
while (token == Token.TYPE_SPECIFIER ||
token == Token.SC_SPECIFIER ||
token == Token.STRUCT || token == Token.UNION ||
token == Token.UPPER_CASE_IDENTIFIER)
parameterDeclaration();
output.indent();
output.endLine(false);
output.skipLine();
compoundStatement();
output.unindent();
output.endLine(false);
output.endPage();
}

// compoundStatement formats a multiple statement block

private void compoundStatement()
{
int noOfDeclarations= 0;
token = lexer.getNextToken();
output.endLine(false);
while (token == Token.TYPE_SPECIFIER ||
token == Token.SC_SPECIFIER ||
token == Token.STRUCT || token == Token.UNION ||
token == Token.UPPER_CASE_IDENTIFIER)
{
declaration();
noOfDeclarations++;
}
if (noOfDeclarations > 0)
```

```
output.skipLine();
while (token != Token.RIGHT_BRACE)
statement();
token = lexer.getNextToken();
output.endLine(false);
}

// statement determines the type of statement and calls the
// appropriate function to format it.

private void statement()
{
if (token == Token.IDENTIFIER)
if ((token = lexer.getNextToken()) == Token.COLON)
token = lexer.getNextToken();
else
{
token = Token.IDENTIFIER;
lexer.putLastToken();
}
switch (token)
{
case LEFT_BRACE:
compoundStatement();
break;
case SWITCH:
switchStatement();
break;
case BREAK:
case CONTINUE:
verifyNextToken(Token.SEMICOLON);
output.endLine(false);
break;
case RETURN:
if ((token = lexer.getNextToken()) != Token.SEMICOLON)
```

```
{
expression(Token.SEMICOLON);
token = lexer.getNextToken();
}
else
token = lexer.getNextToken();
output.endLine(false);
break;
case GOTO:
verifyNextToken(Token.IDENTIFIER);
verifyNextToken(Token.SEMICOLON);
output.endLine(false);
break;
default:
expression(Token.SEMICOLON);
token = lexer.getNextToken();
output.endLine(false);
}
}

// switchStatement formats a switch statement.

private void switchStatement()
{
verifyNextToken(Token.LEFT_PARENTHESIS);
expression(Token.RIGHT_PARENTHESIS);
token = lexer.getNextToken();
output.endLine(false);
output.indent();
verifyCurrentToken(Token.LEFT_BRACE);
output.endLine(false);
while (token == Token.CASE || token == Token.DEFAULT)
{
if (token == Token.CASE)
{
```

```
expression(Token.COLON);
lexer.adjustSpacing(Lexer.SUPPRESS_LEADING_SPACE);
token = lexer.getNextToken();
output.endLine(false);
output.indent();
while (token != Token.CASE && token !=
Token.DEFAULT && token != Token.RIGHT_BRACE)
statement();
output.unindent();
}
else
{
expression(Token.COLON);
lexer.adjustSpacing(Lexer.SUPPRESS_LEADING_SPACE);
token = lexer.getNextToken();
output.endLine(false);
output.indent();
while (token != Token.CASE && token != Token.DEFAULT &&
token != Token.RIGHT_BRACE)
statement();
output.unindent();
}
}
verifyCurrentToken(Token.RIGHT_BRACE);
output.endLine(false);
output.unindent();
}

// externalDeclarations formats external declarations such as
// global variables and function prototypes. It returns if
// it encounters a function heading.

private boolean externalDeclaration()

{
```

```
int braceCount = 0;
boolean indentAtSemicolon = false;
Token lastToken = Token.NOT_FOUND;
while ((braceCount > 0) || (token != Token.SEMICOLON))
{
lexer.checkDeclarationSpacing(token);
if (token == Token.LEFT_BRACE)
{
output.endLine(false);
output.indent();
lastToken = token;
token = lexer.getNextToken();
output.endLine(false);
braceCount++;
}
else if (token == Token.RIGHT_BRACE)
{
lastToken = token;
token = lexer.getNextToken();
indentAtSemicolon = true;
braceCount--;
}
else if (token == Token.LEFT_PARENTHESIS)
{
lastToken = token;
token = lexer.getNextToken();
}
else if (token == Token.RIGHT_PARENTHESIS)
{
lastToken = token;
token = lexer.getNextToken();
if (token != Token.SEMICOLON)
return true;
}
else if (token == Token.ASSIGNMENT_OPERATOR)
```

```
while (token != Token.SEMICOLON)
{
lastToken = token;
token = lexer.getNextToken();
lexer.checkExpressionSpacing(token, lastToken);
}
else if (token == Token.SEMICOLON)
{
lastToken = token;
token = lexer.getNextToken();
if (braceCount > 0)
output.endLine(false);
if (indentAtSemicolon)
{
output.indent();
indentAtSemicolon = false;
}
}
else
{
lastToken = token;
token = lexer.getNextToken();
}
}

token = lexer.getNextToken();
output.endLine(false);
if (indentAtSemicolon)
output.indent();
return false;
}

// parameterDeclaration formats parameter declarations.

private void parameterDeclaration()
```

```
{
int braceCount = 0;
while ((braceCount > 0) || (token != Token.SEMICOLON))
{
lexer.checkDeclarationSpacing(token);
if (token == Token.LEFT_BRACE)
{
output.endLine(false);
output.indent();
token = lexer.getNextToken();
output.endLine(false);
braceCount++;
}
else if (token == Token.RIGHT_BRACE)
{
token = lexer.getNextToken();
output.indent();
braceCount--;
}
else if ((braceCount > 0 ) && (token == Token.SEMICOLON))
{
token = lexer.getNextToken();
output.endLine(false);
}
else
token = lexer.getNextToken();
}
token = lexer.getNextToken();
output.endLine(false);
}

// declaration formats local declarations.

private void declaration()
```

```
{
int braceCount = 0;
boolean indentAtSemicolon = false;

while ((braceCount > 0) || (token != Token.SEMICOLON))
{
lexer.checkDeclarationSpacing(token);
if (token == Token.LEFT_BRACE)
{
output.endLine(false);
output.indent();
token = lexer.getNextToken();
output.endLine(false);
braceCount++;
}
else if (token == Token.RIGHT_BRACE)
{
token = lexer.getNextToken();
indentAtSemicolon = true;
braceCount--;
}
else if (token == Token.SEMICOLON)
{
token = lexer.getNextToken();
if (braceCount > 0)
output.endLine(false);
if (indentAtSemicolon)
{
output.indent();
indentAtSemicolon = false;
}
}
else if (token == Token.ASSIGNMENT_OPERATOR)
expression(Token.SEMICOLON);
```

```
else
token = lexer.getNextToken();
}
token = lexer.getNextToken();
output.endLine(false);
if (indentAtSemicolon)
output.indent();
}

// expression formats an expression. The delimiting token must
// be provided.

private void expression(Token terminator)
{
Token lastToken;

lastToken = Token.NOT_FOUND;
while (token != terminator)
{
lexer.checkExpressionSpacing(token, lastToken);

if (token == Token.LEFT_PARENTHESIS)
{
if (lastToken == Token.IDENTIFIER ||
lastToken == Token.UPPER_CASE_IDENTIFIER)
lexer.adjustSpacing(Lexer.SUPPRESS_LEADING_SPACE);
token = lexer.getNextToken();
expression(Token.RIGHT_PARENTHESIS);
}
lastToken = token;
token = lexer.getNextToken();
}
}

// Gets the next token and then verifies that the supplied token is
```

```
// the required token.

private void verifyNextToken(Token requiredToken)
{
token = lexer.getNextToken();
verifyCurrentToken(requiredToken);
}

// Verifies that the supplied token is the current token.
// Displays an error message if it is not.

private void verifyCurrentToken(Token requiredToken)
{
if (token != requiredToken)
output.outputError("MISSING " + requiredToken.name());
else
token = lexer.getNextToken();

}
}
```

The final class is Output. The singleton object of this class defined in main performs the necessary page formatting of the output. It ensures that each line is no longer than some maximum length and that each page contains no more than a specific number of lines. It also maintains the current indentation level so lines can be properly indented. It also provides the appropriate spacing between tokens.

```
// The Output class controls the formation of lines and pages in the
// output. It also provides explicit functions for controlling the
// indentation and forcing new lines and pages.

import java.io.*;

class Output
{
```

```
private static final int INDENT_INCREMENT = 4, LEFT_MARGIN = 0,
LINES_PER_PAGE = 56, HEADING_LENGTH = 70,
CHARACTERS_PER_LINE = 78;
private PrintWriter file;
private int linesOnPage;
private int pageNumber;
private int indentation;
private String buffer = "";
private String heading;

// The constructor initializes the private instance variables.
// It constructs a page heading containing the input file name.

public Output(String fileName)
throws FileNotFoundException, IOException
{
file = new PrintWriter(new FileWriter(fileName + "_.c"));
linesOnPage = LINES_PER_PAGE;
pageNumber = 1;
indentation = LEFT_MARGIN;
heading = fileName;
for (int i = 0; i < HEADING_LENGTH - fileName.length(); i++)
heading += ' ';
}

// Closes the output file.

public void close()
{
file.close();
}

// outputToken outputs the token string, adjusting spacing
// specified by the spacing word.
```

```
public void outputToken(String token, int spacing)
{
if (buffer.length() + token.length() > CHARACTERS_PER_LINE)
{
outputLine(buffer);
buffer = "";
}
if ((spacing & Lexer.SUPPRESS_LEADING_SPACE) != 0)
if (buffer.length() > 0 &&
buffer.charAt(buffer.length() - 1) == ' ')
buffer = buffer.substring(0, buffer.length() - 1);
buffer += token;
if ((spacing & Lexer.SUPPRESS_TRAILING_SPACE) == 0)
buffer += ' ';
}

// outputDirective prints out a compiler directive starting at the
// left margin.

public void outputDirective(String directive)
{
outputLine(directive);
}

// outputError prints out error messages.

public void outputError(String error)
{
file.println(error);
}

// indent increments the indentation variable.

public void indent()
{
```

```
indentation += INDENT_INCREMENT;
}

// unindent decrements the indentation variable.

public void unindent()
{
indentation -= INDENT_INCREMENT;
}

// endLine calls outputLine to write out a line. If the parameter
// forceNewPage is true, then newPage is called.

public void endLine(boolean forceNewPage)
{
if (forceNewPage && (linesOnPage > 0))
newPage();
if (buffer != "")
outputLine(buffer);
buffer = "";
}

// skipLine skips a line.

public void skipLine()
{
outputLine("");
}

// endPage sets linesOnPage to force a call to newPage.

public void endPage()
{
linesOnPage = LINES_PER_PAGE;
}
```

```
// newPage does a form feed and prints a new page heading.

public void newPage()
{
file.println("\f" + heading + "PAGE " + pageNumber++);
linesOnPage = 0;
}

// outputLine fills up the number of spaces in the margin
// and prints a line from the buffer.
// It then increments lines per page.

public void outputLine(String line)
{
String buffer = "";

if (linesOnPage >= LINES_PER_PAGE)
newPage();
for (int space = 0; space < LEFT_MARGIN + indentation; space++)
buffer += ' ';
buffer += line;
file.println(buffer);
linesOnPage++;
}
}
```